

PROJECT BRICK WARFARE

AI 활용 기술 문서

1. 문서 목적

이 문서는 PROJECT BRICK WARFARE 제작 과정에서 사용한 생성형 AI 도구, 주요 프롬프트, AI 산출물의 검토 방식과 게임 내부 AI 구조를 설명합니다.

게임 제작에는 OpenAI Codex를 활용했습니다. 완성된 게임은 별도 AI API, 서버, 유료 모델 없이 GitHub Pages의 정적 웹 빌드만으로 실행됩니다.

2. AI 활용 요약

구분	활용 내용
생성형 AI 도구	OpenAI Codex
기획	대규모 전쟁 시뮬레이터 아이디어를 3분 웹 게임으로 축소
설계	TypeScript / Three.js 모듈 구조, 상태 흐름, 성능 예산 설계
구현	유닛, 전투, 점령, AI, 파괴, HUD, 오디오 코드 작성 보조
검증	타입 검사, 프로덕션 빌드, Chromium 상호작용 테스트
문서	게임 설명서, AI 활용서, 촬영 체크리스트 작성 보조
런타임 외부 AI	사용하지 않음

3. 생성형 AI 활용 과정

3.1 기획 축소

초기 아이디어는 무한 맵, 대규모 병력, 다수 항공기, 극실사 물리, 실시간 외교를 모두 포함했습니다. 생성형 AI와의 반복 검토를 통해 심사자가 짧은 시간 안에 핵심을 이해할 수 있도록 기본 모드를 다음과 같이 축소했습니다.

- 3분 경기
- 중립 거점 A/B/C
- 진영별 시작 병력 10기와 공중 전력
- God Mode와 직접 조종의 즉시 전환
- 100점 선취
- 1:1 기본, 1:1:1 선택
- Sandbox에 확장 기능 보존

AI가 제안한 범위를 그대로 적용하지 않고, 실제 브라우저 프레임·조작 난이도·첫 30초 가독성을 기준으로 사람이 최종 범위를 결정했습니다.

3.2 주요 프롬프트와 반영 내역

아래 문장은 실제 개발 대화의 핵심 지시를 짧게 정리한 것입니다.

주요 프롬프트 / 지시	AI 활용 결과	사람의 검토·수정
“God Mode와 모든 유닛 직접 조종을 오갈 수 있는 블록 전쟁 게임을 구현해라.”	이중 카메라·입력 상태와 유닛 방의 구조 초안	Challenge에서는 아군만, Sandbox에서는 모든 진영으로 규칙 분리
“거점 점령이 1순위이며 5초 동안 가장 많은 병력이 있으면 점령한다.”	거점 단독 우세 판정과 점령 진행도 구현	동률 시 중단, HUD 교전 표시, 지형 진영색 갱신 추가
“유닛이 건물과 장벽에 걸리지 말고 우회해야 한다.”	구조물 선분 충돌과 우회 코너 탐색 제안	정지 감지와 좌우 탈출 경로를 추가하고 실제 브라우저에서 이동 확인
“보병 외 유닛은 일반공격과 강한 특수공격을 사용한다.”	병과별 무기 데이터와 재사용 대기시간 분리	일반탄 가시성, 특수탄 건물 파괴력, 드론 자폭을 직접 조정
“총알과 폭탄은 십자선 정중앙으로 직선 비행한다.”	카메라 중앙 레이캐스트 기반 조준점 계산	소유 유닛 자체 충돌 제외, 지형·구조물·적 유닛 우선순위 검증
“렉과 CPU 사용량을 줄여라.”	AI 주기 분리, 인스턴싱, 오브젝트 상한 제안	Challenge 진영별 10기, AI 15 Hz, 파편 상한, 소프트웨어 저해상도 적용
“예선을 통과할 수 있도록 완전한 게임으로 만들어라.”	3분 점수전, 결과 화면, 제출 문서 구조 제안	전체 경기 자동 완주 테스트 후 실제 규칙에 맞게 문서 재작성

3.3 코드 생성과 리뷰 원칙

생성형 AI가 만든 코드는 다음 기준으로 검토했습니다.

- 1. 기존 모듈 경계를 유지하고 한 파일에 기능을 몰아넣지 않습니다.
- 2. TypeScript 타입 검사에서 오류가 없어야 합니다.
- 3. 외부 유료 API가 없을 때도 모든 핵심 기능이 동작해야 합니다.
- 4. 사용자 지시와 실제 조작이 다르면 코드 계산 방향까지 확인합니다.
- 5. 프레임 저하가 경기 타이머를 느리게 만들지 않아야 합니다.
- 6. 문서에 구현되지 않은 기능을 구현된 것처럼 기재하지 않습니다.

4. 게임 내부 AI

게임 내부 AI는 LLM이 아니라 브라우저에서 실행되는 경량 규칙·효용·가중치 시스템입니다. 정적 GitHub Pages에서도 지연과 비용 없이 반복 플레이가 가능하도록 선택했습니다.

4.1 전술 유닛 AI

BattlefieldAI 는 각 유닛에 대해 다음 순서로 행동을 결정합니다.

직접 조종 여부 확인

- 플레이어가 내린 명령 확인
- 점령해야 할 거점 배정
- 가까운 위협 탐색
- 사선 확인
- 이동 중 일반/특수공격
- 구조물 우회
- 정지 상태가 길면 탈출 경로 재계산

거점이 1순위 목표입니다. 멀리 있는 적을 추격하느라 점령을 포기하지 않도록, 현재 거점 주변의 위협이나 바로 가까운 적만 교전합니다.

공중 유닛도 거점 목표를 공유합니다.

- 드론과 헬기: 거점 반경에서 낮게 체공
- 전투기: 더 넓은 제공권 반경에서 순회
- 드론: 거점 주변 32m 이내의 적에게 자율 자폭 가능

Challenge의 초기 병력에는 진영별 헬기 1기와 전투기 1기가 포함됩니다. 손실 병력은 살아 있는 본진과 점령 거점을 순환하는 증원 지점에서 보충되며, 지휘관 전략에 따라 보병·전차·드론·헬기·전투기 중 병과가 결정됩니다.

전장은 시가전·능선전·참호전의 세 변형 중 하나를 경기 시작 시 선택합니다. 각 변형은 같은 A/B/C 목표 구조를 유지하면서 지형 높이, 참호, 건물, 방벽과 거점 좌표를 달리합니다.

4.2 내비게이션 AI

NavigationSystem은 완전한 격자 경로 탐색 대신 현재 규모에 맞는 경량 국소 회피를 사용합니다.

1. 현재 위치와 목적지를 잇는 선분이 구조물 로컬 경계와 교차하는지 검사합니다.
2. 막는 구조물의 네 모서리 후보를 계산합니다.
3. 유닛별 좌우 선호와 전체 경로 길이를 합산해 우회점을 선택합니다.
4. 1.35초 이상 실질적으로 움직이지 못하면 측면 탈출점을 새로 생성합니다.

이 방식은 3분짜리 소규모 전장에서 A* 탐색용 내비게이션 메시를 생성하지 않고도 건물과 장벽을 우회할 수 있게 합니다.

4.3 적응형 적 지휘관

AdaptiveDirector는 플레이어가 어느 거점에 어떤 병과를 반복해서 명령하는지 가중치로 기록합니다.

- 목적 거점
- 명령받은 병과
- 시간 감쇠가 적용된 거점별 가중치
- 현재 최고 확률 목표
- 예측 신뢰도

가중치를 확률로 정규화해 HUD에 예상 목표와 신뢰도를 표시합니다. 신뢰도가 높아지면 적 예비대 일부가 예상 거점으로 이동합니다.

플레이어가 높은 신뢰도로 예측된 목표와 다른 거점에 명령하면 기만 성공으로 판정합니다. 적은 직전 예상 목표에 잠시 묶이고 플레이어는 5점을 얻습니다.

가중치는 `localStorage` 에 저장될 수 있으나, 개인 식별 정보나 네트워크 전송은 없습니다. 이 기능은 온라인 학습 모델이 아니라 플레이 패턴에 즉시 적응하는 투명한 가중치 시스템입니다.

4.4 진영 지휘관

각 진영은 일정 주기로 다음 값을 평가합니다.

- 생존 병력 수
- 전차와 항공 전력
- 소유 거점 수
- 진영 고유 교리

평가 결과에 따라 **거점 확보**, **집중 돌파**, **방어**, **항공 우세**, **참호 방어** 중 하나를 선택합니다. 선택한 작전은 증원 병과와 우선 목표에 반영되며 변경 이유가 HUD 알림으로 표시됩니다.

4.5 Sandbox 자율 외교

Sandbox에서는 세 진영의 생존 체력, 유닛 가치, 거점 가치를 합산해 전력 균형을 평가합니다. 한 진영이 과도하게 강해지면 나머지 진영이 동맹 또는 휴전을 선택할 수 있습니다. 관계는 항상 양쪽에 대칭 적용됩니다.

Challenge에서는 명확한 경쟁 규칙을 위해 외교를 비활성화합니다.

5. 전투·물리 계산

5.1 직선 발사체

모든 일반탄과 특수탄은 발사 시 계산한 방향 벡터를 유지합니다. 중력과 바람으로 휘지 않으며 카메라 중앙 십자선이 가리킨 레이캐스트 충돌점으로 발사됩니다.

AI는 발사 전에 지형 높이를 구간 샘플링합니다. 산이나 능선이 사선을 막으면 발사를 반복하지 않고 거점 방향으로 계속 이동합니다.

5.2 장갑과 도탄

피격 방향과 유닛 전방 벡터의 내적을 이용해 전면·측면·후면 장갑 계수를 결정합니다. 입사각이 나쁘고 관통력이 부족하면 도탄으로 처리합니다.

5.3 특수공격

- 전차: 철갑 주포
- 전투기: 공대지 미사일
- 헬기: 중형 로켓
- 드론: 돌입 자폭

특수공격은 일반공격보다 재사용 대기시간이 길지만 피해량·관통력·폭발 반경이 커서 구조물을 한 번에 붕괴시킬 수 있습니다.

5.4 블록 파괴

건물 블록은 재질별 `InstancedMesh` 로 렌더링하면서 각 인스턴스의 체력과 지지 여부를 별도 관리합니다.

- 피해 반경 안 블록만 손상

- 파괴된 인스턴스 행렬을 0 크기로 변경
- 파괴된 블록 일부를 물리 파편으로 전환
- 하부 지지 블록 생존 비율이 48% 아래면 상부 연쇄 붕괴

유닛과 나무도 실제로 파괴된 시점에만 블록 파편을 생성합니다. 단순 폭발 효과만으로 멀쩡한 대상이 분해되지 않습니다.

6. 생성형 오디오

외부 음원 파일을 사용하지 않습니다. **BattleAudio** 가 Web Audio API의 오실레이터, 노이즈 버퍼, 필터와 게인 곡선을 조합해 다음 소리를 런타임에 합성합니다.

- 일반 사격
- 특수공격
- 폭발
- 명령 확인
- 빙의 연결·해제
- 거점 점령
- 승리·패배
- 전투 강도에 따라 속도가 바뀌는 배경 펄스

따라서 음원 저작권 출처가 별도로 필요하지 않습니다.

7. 성능 최적화

항목	적용 내용
병력	Challenge 진영별 10기
AI	Challenge 15 Hz
HUD	5 Hz
거점 판정	10 Hz
지형	5×5 청크
발사체	최대 72개
구조물 잔해	구조물별 최대 48개
장식 파편	전역 최대 104개
구조물 렌더	재질별 InstancedMesh
그림자	Challenge 비활성화
소프트웨어 렌더	내부 픽셀 비율 0.68

경기 시간은 렌더링 델타와 분리했습니다. 낮은 프레임에서도 물리 시뮬레이션은 안전한 작은 간격으로 진행하고, 경기 타이머는 실제 경과 시간에 맞춰 감소합니다.

8. 검증 기록

다음 검증을 수행했습니다.

- 1. `npm run check` TypeScript 프로젝트 검사
- 2. `npm run build` Vite 프로덕션 빌드
- 3. Chromium에서 시작 화면 렌더링
- 4. 작전 시작과 포인터 잠금
- 5. 숫자키 거점 명령
- 6. Enter 직접 조종과 G 복귀
- 7. 일반·특수공격 입력
- 8. 거점 점령, 점수 증가, 제한 시간 종료
- 9. 1:1과 1:1:1 HUD
- 10. 결과 화면과 재도전
- 11. 브라우저 콘솔 오류 확인

전체 흐름을 빠르게 검증하기 위한 `?test=1` 개발 쿼리는 경기 시간을 45초, 목표 점수를 20점으로 변경합니다. 기본 링크에는 적용되지 않으며 실제 제출 빌드는 3분·100점 규칙입니다.

9. 외부 에셋과 오픈소스

외부 에셋

- 이미지: 사용하지 않음
- 3D 모델: 사용하지 않음
- 음원: 사용하지 않음
- 폰트 파일: 사용하지 않음

게임 화면의 유닛, 구조물, 지형, 식생, 발사체, 파편과 UI 효과는 모두 코드에서 생성합니다.

오픈소스

라이브러리	용도	라이선스
Three.js	WebGL 3D 렌더링	MIT
Vite	개발 서버와 프로덕션 번들	MIT
TypeScript	정적 타입 검사와 컴파일	Apache-2.0

10. 한계와 정직한 표기

- 게임 내부 적응형 지휘관은 LLM이나 딥러닝 모델이 아닙니다.
- 탄도와 항공 현상은 군사 훈련용이 아닌 게임 플레이용 축약 모델입니다.
- Challenge의 지형과 항공 물리는 짧은 경기와 브라우저 성능에 맞춘 축약 모델입니다.

- 생성형 AI 산출물은 타입 검사와 브라우저 테스트를 거쳐 사람이 최종 채택했습니다.